

2.1.6 Aplicações de rede abordadas neste livro

Novas aplicações de Internet de domínio público e proprietárias são desenvolvidas todos os dias. Em vez de tratarmos de um grande número dessas aplicações de maneira enciclopédica, preferimos focalizar um pequeno número de aplicações ao mesmo tempo importantes e populares. Neste capítulo, discutiremos cinco aplicações populares: a Web, a transferência de arquivos, o correio eletrônico, o serviço de diretório e aplicações P2P. Discutiremos primeiramente a Web não somente porque ela é uma aplicação imensamente popular, mas também porque seu protocolo de camada de aplicação, HTTP, é direto e fácil de entender. Após examinarmos a Web, examinaremos brevemente o FTP porque este protocolo oferece um ótimo contraste com o HTTP. Em seguida, discutiremos o correio eletrônico, a primeira aplicação de enorme sucesso da Internet. O correio eletrônico é mais complexo do que a Web, no sentido de que usa não somente um, mas vários protocolos de camada de aplicação. Após o e-mail, estudaremos o DNS, que provê um serviço de diretório para a Internet. A maioria dos usuários não interage diretamente com o DNS; em vez disso, eles o chamam indiretamente por meio de outras aplicações (inclusive a Web, a transferência de arquivos e o correio eletrônico). O DNS ilustra primorosamente como um componente de funcionalidade de núcleo de rede (tradução de nome de rede para endereço de rede) pode ser implementado na camada de aplicação da Internet. Finalmente, discutiremos o compartilhamento de arquivos P2P que, segundo algumas medições (por exemplo, tráfego de rede), é a classe de aplicações mais popular da Internet de hoje. Finalmente, discutiremos várias aplicações P2P, incluindo distribuição de arquivo, banco de dados distribuídos e telefonia IP.

2.2 A Web e o HTTP

Até a década de 1990, a Internet era usada primordialmente por pesquisadores, acadêmicos e estudantes universitários para se interligar com hospedeiros remotos, transferir arquivos de hospedeiros locais para hospedeiros remotos e vice-versa, enviar e receber notícias e enviar e receber correio eletrônico. Embora essas aplicações fossem (e continuem a ser) extremamente úteis, a Internet não era conhecida fora das comunidades acadêmicas e de pesquisa. Então, no início da década de 1990, entrou em cena uma nova aplicação importantíssima — a World Wide Web [Berners-Lee, 1994]. A Web é a aplicação da Internet que chamou a atenção do público em geral. Ela transformou drasticamente a maneira como pessoas interagem dentro e fora de seus ambientes de trabalho. Alçou a Internet de apenas mais uma entre muitas redes de dados para, essencialmente, a única rede de dados.

Talvez o que mais atraia a maioria dos usuários da Web é que ela funciona *por demanda*. Usuários recebem o que querem, quando querem, o que é diferente da transmissão de rádio e de televisão, que força o usuário a sintonizar quando o provedor disponibiliza o conteúdo. Além de funcionar por demanda, a Web tem muitas outras características maravilhosas que as pessoas adoram. É muitíssimo fácil para qualquer indivíduo fazer com que informações fiquem disponíveis na Web; todo mundo pode se transformar em editor a um custo extremamente baixo. Hiperenlaces e buscadores nos ajudam a navegar pelo oceano dos sites Web. Dispositivos gráficos estimulam nossos sentidos. Formulários, applets Java e muitos outros dispositivos nos habilitam a interagir com páginas e sites. E, cada vez mais, a Web oferece um menu de interfaces para vastas quantidades de material de vídeo e áudio armazenadas na Internet — áudio e vídeo que podem ser acessados por demanda.

2.2.1 Descrição geral do HTTP

O **HTTP — Protocolo de Transferência de Hipertexto** (HyperText Transfer Protocol) — protocolo de camada de aplicação da Web, está no coração da Web e é definido no [RFC 1945] e no [RFC 2616]. O HTTP é implementado em dois programas: um programa cliente e outro servidor. Os dois programas, executados em sistemas finais diferentes, conversam um com o outro por meio da troca de mensagens HTTP. O HTTP define a

estrutura dessas mensagens e o modo como o cliente e o servidor as trocam. Antes de explicarmos detalhadamente o HTTP, devemos revisar a terminologia da Web.

Uma **página Web** (também denominada documento) é constituída de objetos. Um **objeto** é simplesmente um arquivo — tal como um arquivo HTML, uma imagem JPEG, um applet Java, ou um clipe de vídeo — que se pode acessar com um único URL. A maioria das páginas Web é constituída de um **arquivo-base HTML** e diversos objetos referenciados. Por exemplo, se uma página Web contiver um texto HTML e cinco imagens JPEG, então ela terá seis objetos: o arquivo-base HTML e mais as cinco imagens. O arquivo-base HTML referencia os outros objetos na página com os URLs dos objetos. Cada URL tem dois componentes: o nome do hospedeiro do servidor que abriga o objeto e o nome do caminho do objeto. Por exemplo, no URL

```
http://www.someSchool.edu/someDepartment/picture.gif
```

`www.someSchool.edu` é o nome do hospedeiro e `/someDepartment/picture.gif` é o do caminho. Como browsers Web também implementam o lado cliente do HTTP, podemos usar as palavras *browser* e *cliente* indiferentemente no contexto da Web. Os servidores Web também implementam o lado servidor do HTTP, abrigam objetos Web, cada um endereçado por um URL. São servidores Web populares o Apache e o Microsoft Internet Information Server.

O HTTP define como clientes Web requisitam páginas Web aos servidores e como eles as transferem a clientes. Discutiremos detalhadamente a interação entre cliente e servidor mais adiante, mas a ideia geral está ilustrada na Figura 2.6. Quando um usuário requisita uma página Web (por exemplo, clica sobre um hiperenlace), o browser envia ao servidor mensagens de requisição HTTP para os objetos da página. O servidor recebe as requisições e responde com mensagens de resposta HTTP que contêm os objetos.

O HTTP usa o TCP como seu protocolo de transporte subjacente (em vez de rodar em UDP). O cliente HTTP primeiramente inicia uma conexão TCP com o servidor. Uma vez estabelecida a conexão, os processos do browser e do servidor acessam o TCP por meio de suas interfaces sockets. Como descrito na Seção 2.1, no lado cliente a interface socket é a porta entre o processo cliente e a conexão TCP; no lado servidor, ela é a porta entre o processo servidor e a conexão TCP.

O cliente envia mensagens de requisição HTTP para sua interface socket e recebe mensagens de resposta HTTP de sua interface socket. De maneira semelhante, o servidor HTTP recebe mensagens de requisição de sua interface socket e envia mensagens de resposta para sua interface socket. Assim que o cliente envia uma mensagem para sua interface socket, a mensagem sai de suas mãos e “passa para as mãos do TCP”. Lembre-se de que na Seção 2.1 dissemos que o TCP provê ao HTTP um serviço confiável de transferência de dados, o que implica que toda mensagem de requisição HTTP emitida por um processo cliente chegará intacta ao servidor. De maneira semelhante, toda mensagem de resposta HTTP emitida pelo processo servidor chegará intacta ao cliente. Percebemos, nesse ponto, uma das grandes vantagens de uma arquitetura de camadas — o



Figura 2.6 Comportamento de requisição-resposta do HTTP

HTTP não precisa se preocupar com dados perdidos ou com detalhes de como o TCP recupera a perda de dados ou os reordena dentro da rede. Essa é a tarefa do TCP e dos protocolos das camadas mais inferiores da pilha de protocolos.

É importante notar que o servidor envia ao cliente os arquivos solicitados sem armazenar nenhuma informação de estado sobre este. Se um determinado cliente solicita o mesmo objeto duas vezes em um período de poucos segundos, o servidor não responde dizendo que acabou de enviar o objeto ao cliente; em vez disso, envia novamente o objeto, pois já esqueceu completamente o que fez antes. Como o servidor HTTP não mantém nenhuma informação sobre clientes, o HTTP é denominado um **protocolo sem estado**.

Salientamos também que a Web usa a arquitetura de aplicação cliente-servidor, como descrito na Seção 2.1. Um servidor Web está sempre em funcionamento, tem um endereço IP fixo e atende requisições de potencialmente milhões de browsers diferentes.

2.2.2 Conexões persistentes e não persistentes

Em muitas aplicações da Internet, o cliente e o servidor se comunicam por um período prolongado de tempo, em que o cliente faz uma série de requisições e o servidor responde a cada uma delas. Dependendo da aplicação e de como ela está sendo usada, a série de requisições pode ser feita de forma consecutiva, periodicamente em intervalos regulares ou esporadicamente. Quando a interação cliente-servidor acontece por meio de conexão TCP, o criador da aplicação precisa tomar uma importante decisão — cada par de requisição/resposta deve ser enviado por uma conexão TCP *distinta* ou todas as requisições e suas respostas devem ser enviadas por uma *mesma* conexão TCP? Na abordagem anterior, a aplicação utiliza conexões não persistentes; e na última abordagem, conexões persistentes. Para obter uma maior compreensão deste assunto, vamos analisar as vantagens e desvantagens das conexões não persistentes e das conexões persistentes no contexto de uma aplicação específica, o HTTP, que pode utilizar essas duas conexões. Embora o HTTP utilize conexões persistentes em seu modo padrão, os clientes e servidores HTTP podem ser configurados para utilizar a conexão não persistente.

O HTTP com conexões não persistentes

Vamos percorrer as etapas da transferência de uma página Web de um servidor para um cliente para o caso de conexões não persistentes. Vamos supor que uma página consista em um arquivo-base HTML e em dez imagens JPEG e que todos esses 11 objetos residam no mesmo servidor. Suponha também que o URL para o arquivo-base HTTP seja

```
http://www.someSchool.edu/someDepartment/home.index
```

Eis o que acontece:

1. O processo cliente HTTP inicia uma conexão TCP para o servidor `www.someSchool.edu` na porta número 80, que é o número de porta default para o HTTP. Associados à conexão TCP, haverá um socket no cliente e um socket no servidor.
2. O cliente HTTP envia uma mensagem de requisição HTTP ao servidor através de seu socket. Essa mensagem inclui o nome de caminho `/someDepartment/home.index`. (Discutiremos mensagens HTTP mais detalhadamente logo adiante.)
3. O processo servidor HTTP recebe a mensagem de requisição através de seu socket, extrai o objeto `/someDepartment/home.index` de seu armazenamento (RAM ou disco), encapsula o objeto em uma mensagem de resposta HTTP e a envia ao cliente através de seu socket.
4. O processo servidor HTTP ordena ao TCP que encerre a conexão TCP. (Mas, na realidade, o TCP só encerrará a conexão quando tiver certeza de que o cliente recebeu a mensagem de resposta intacta.)
5. O cliente HTTP recebe a mensagem de resposta e a conexão TCP é encerrada. A mensagem indica que o objeto encapsulado é um arquivo HTML. O cliente extrai o arquivo da mensagem de resposta, analisa o arquivo HTML e encontra referências aos dez objetos JPEG.
6. As primeiras quatro etapas são repetidas para cada um dos objetos JPEG referenciados.

À medida que recebe a página Web, o browser a apresenta ao usuário. Dois browsers diferentes podem interpretar (isto é, exibir ao usuário) uma página Web de modos ligeiramente diferentes. O HTTP não tem nada a ver com o modo como uma página Web é interpretada por um cliente. As especificações do HTTP [RFC 1945] e [RFC 2616] definem apenas o protocolo de comunicação entre o programa cliente HTTP e o programa servidor HTTP.

As etapas apresentadas ilustram a utilização de conexões não persistentes, nas quais cada conexão TCP é encerrada após o servidor enviar o objeto — a conexão não persiste para outros objetos. Note que cada conexão TCP transporta exatamente uma mensagem de requisição e uma mensagem de resposta. Assim, nesse exemplo, quando um usuário solicita a página Web, são geradas 11 conexões TCP.

Nas etapas descritas, fomos intencionalmente vagos sobre se os clientes obtêm as dez JPEGs por meio de dez conexões TCP em série ou se algumas delas são recebidas por conexões TCP paralelas. Na verdade, usuários podem configurar browsers modernos para controlar o grau de paralelismo. Nos modos default, a maioria dos browsers abre de cinco a dez conexões TCP paralelas e cada uma delas manipula uma transação requisição/resposta. Se o usuário preferir, o número máximo de conexões paralelas poderá ser fixado em um, caso em que as dez conexões são estabelecidas em série. Como veremos no próximo capítulo, a utilização de conexões paralelas reduz o tempo de resposta.

Antes de continuarmos, vamos fazer um cálculo simples para estimar o tempo que transcorre entre a requisição e o recebimento de um arquivo-base HTTP por um cliente. Para essa finalidade, definimos o **tempo de viagem de ida e volta** (*round-trip time* — RTT), ou seja, o tempo que leva para um pequeno pacote viajar do cliente ao servidor e de volta ao cliente. O RTT inclui atrasos de propagação de pacotes, de fila de pacotes em roteadores e comutadores intermediários e de processamento de pacotes. (Esses atrasos foram discutidos na Seção 1.4.) Considere, agora, o que acontece quando um usuário clica sobre um hiperenlace. Como ilustrado na Figura 2.7, isso faz com que o browser inicie uma conexão TCP entre ele e o servidor Web, o que envolve uma ‘apresentação de três vias’ — o cliente envia um pequeno segmento TCP ao servidor, o servidor o reconhece e responde com um pequeno segmento ao cliente que, por fim, o reconhece novamente para o servidor. As duas primeiras partes da apresentação de três vias representam um RTT. Após concluir essas partes, o cliente envia a mensagem de requisição HTTP combinada com a terceira parte da apresentação de três vias (o reconhecimento) para dentro da conexão TCP. Assim que a mensagem de requisição chega ao servidor, este envia o arquivo HTML por meio

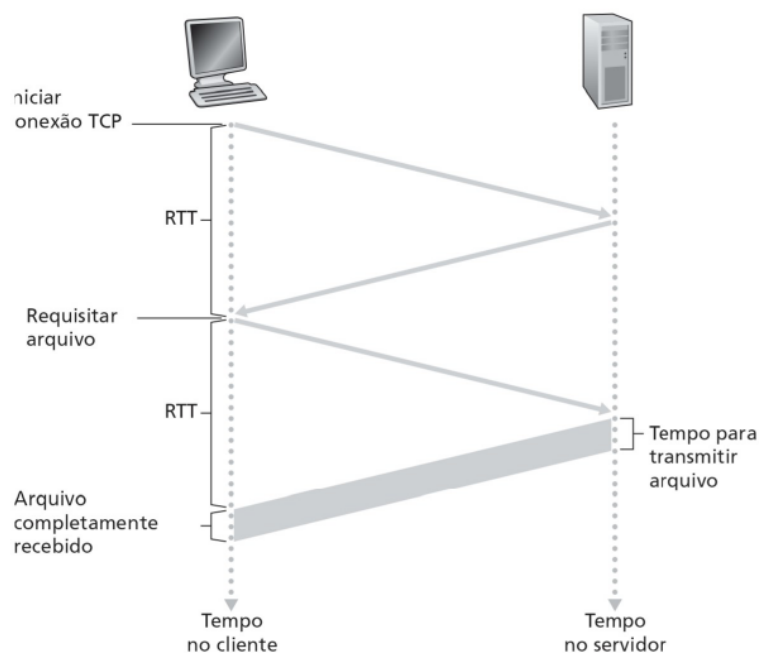


Figura 2.7 Cálculo simples para o tempo necessário para solicitar e receber um arquivo HTML

da conexão TCP. Essa requisição/resposta HTTP causa mais um RTT. Assim, aproximadamente, o tempo total de resposta são dois RTTs mais o tempo de transmissão do arquivo HTML no servidor.

O HTTP com conexões persistentes

Conexões não persistentes têm algumas desvantagens. Em primeiro lugar, uma nova conexão deve ser estabelecida e mantida para *cada objeto solicitado*. Para cada uma dessas conexões, devem ser alocados buffers TCP e conservadas variáveis TCP tanto no cliente quanto no servidor. Isso pode sobrecarregar seriamente o servidor Web, que poderá estar processando requisições de centenas de diferentes clientes ao mesmo tempo. Em segundo lugar, como acabamos de descrever, cada objeto sofre dois RTTs — um RTT para estabelecer a conexão TCP e um RTT para solicitar e receber um objeto.

Em conexões persistentes, o servidor deixa a conexão TCP aberta após enviar resposta. Requisições e respostas subsequentes entre os mesmos cliente e servidor podem ser enviadas por meio da mesma conexão. Em particular, uma página Web inteira (no exemplo anterior, o arquivo-base HTML e as dez imagens) pode ser enviada mediante uma única conexão TCP persistente. Essas requisições por objetos podem ser feitas consecutivamente sem ter de esperar por respostas a requisições pendentes (paralelismo). Normalmente, o servidor HTTP fecha uma conexão quando ela não é usada durante um certo tempo (um intervalo de pausa configurável). Quando o servidor recebe requisições consecutivas, os objetos são enviados de forma ininterrupta.

O modo default do HTTP usa conexões persistentes com paralelismo. Faremos uma comparação quantitativa entre os desempenhos de conexões persistentes e não persistentes nos exercícios de fixação dos capítulos 2 e 3. Aconselhamos o leitor interessado a consultar [Heidemann, 1997; Nielsen, 1997].

2.2.3 Formato da mensagem HTTP

As especificações do HTTP [RFC 2616] definem os formatos das mensagens HTTP. Há dois tipos de mensagens HTTP: de requisição e de resposta. Ambas serão discutidas a seguir.

Mensagem de requisição HTTP

Apresentamos a seguir uma mensagem de requisição HTTP típica:

```
GET /somedir/page.html HTTP/1.1
Host: www.someschool.edu
Connection: close
User-agent: Mozilla/4.0
Accept-language: fr
```

Podemos aprender bastante examinando essa simples mensagem de requisição. Em primeiro lugar, vemos que ela está escrita em texto ASCII comum, de modo que pode ser lida por qualquer um que conheça computadores. Em segundo lugar, vemos que ela é constituída de cinco linhas, cada uma seguida de um ‘carriage return’ e ‘line feed’ (fim de linha) para o início de uma nova linha. A última linha é seguida de um comando adicional de ‘carriage return’ e ‘line feed’. Embora essa mensagem de requisição específica tenha cinco linhas, uma mensagem de requisição pode ter muito mais do que isso e também menos do que isso, podendo conter até mesmo uma única linha. A primeira linha de uma mensagem de requisição HTTP é denominada **linha de requisição**; as subsequentes são denominadas **linhas de cabeçalho**. A linha de requisição tem três campos: o campo do método, o do URL e o da versão do HTTP. O campo do método pode assumir vários valores diferentes, entre eles GET, POST e HEAD. A grande maioria das mensagens de requisição HTTP emprega o método GET, o qual é usado quando o browser requisita um objeto e este é identificado no campo do URL. Nesse exemplo, o browser está requisitando o objeto /somedir/page.html. A versão é autoexplicativa. Nesse exemplo, o browser implementa a versão HTTP/1.1.

Vamos agora examinar as linhas de cabeçalho do exemplo. A linha de cabeçalho `Host: www.some-school.edu` especifica o hospedeiro no qual o objeto reside. Talvez você ache que ela é desnecessária, pois já existe uma conexão TCP para o hospedeiro. Mas, como veremos na Seção 2.2.5, a informação fornecida pela linha de cabeçalho do hospedeiro é requerida por armazenadores intermediários da Web. Ao incluir a linha de cabeçalho `Connection: close`, o browser está dizendo ao servidor que não quer usar conexões persistentes; quer que o servidor feche a conexão após o envio do objeto requisitado. A linha de cabeçalho `User-agent:` especifica o agente de usuário, isto é, o tipo de browser que está fazendo a requisição ao servidor. No caso em questão, o agente de usuário é o Mozilla/4.0, um browser da Netscape. Essa linha de cabeçalho é útil porque, na verdade, o servidor pode enviar versões diferentes do mesmo objeto a tipos diferentes de agentes de usuário. (Cada uma das versões é endereçada pelo mesmo URL.) Por fim, o cabeçalho `Accept-language:` mostra que o usuário prefere receber uma versão em francês do objeto se esse objeto existir no servidor; se não existir, o servidor deve enviar a versão default. O cabeçalho `Accept-language:` é apenas um dos muitos cabeçalhos de negociação de conteúdo disponíveis no HTTP.

Acabamos de examinar um exemplo. Vamos agora analisar o formato geral de uma mensagem de requisição, ilustrado na Figura 2.8.

Vemos que o formato geral de uma mensagem de requisição é muito parecido com nosso exemplo anterior. Contudo, você provavelmente notou que, após as linhas de cabeçalho (e após a linha adicional com 'carriage return' e 'line feed'), há um 'corpo de entidade'. O corpo de entidade fica vazio com o método GET, mas é utilizado com o método POST. Um cliente HTTP normalmente usa o método POST quando o usuário preenche um formulário — por exemplo, quando fornece palavras de busca a um site buscador. Com uma mensagem POST, o usuário continua solicitando uma página Web ao servidor, mas o conteúdo específico dela depende do que o usuário escreveu nos campos do formulário. Se o valor do campo de método for POST, então o corpo de entidade conterá o que o usuário digitou nos campos do formulário.

Seríamos omissos se não mencionássemos que uma requisição gerada com um formulário não utiliza necessariamente o método POST. Ao contrário, formulários HTML frequentemente utilizam o método GET e incluem os dados digitados (nos campos do formulário) no URL requisitado. Por exemplo, se um formulário usar o método GET, tiver dois campos e as entradas desses dois campos forem `monkeys` e `bananas`, então a estrutura do URL será `www.somesite.com/animalsearch?monkeys&bananas`. Ao navegar normalmente pela Web, você provavelmente já notou URLs extensos como esse.

O método HEAD é semelhante ao GET. Quando um servidor recebe uma requisição com o método HEAD, responde com uma mensagem HTTP, mas deixa de fora o objeto requisitado. Esse método é frequentemente usado pelos desenvolvedores de servidores HTTP para depuração.

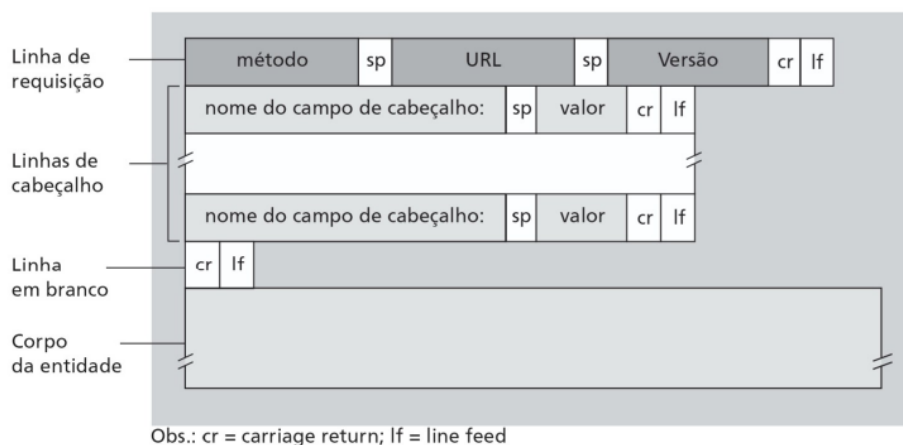


Figura 2.8 Formato geral de uma mensagem de requisição

O método PUT é frequentemente usado juntamente com ferramentas de edição da Web. Permite que um usuário carregue um objeto para um caminho específico (diretório) em um servidor Web específico. O método PUT também é usado por aplicações que precisam carregar objetos para servidores Web. O método DELETE permite que um usuário, ou uma aplicação, elimine um objeto em um servidor Web.

Mensagem de resposta HTTP

Apresentamos a seguir uma mensagem de resposta HTTP típica. Essa mensagem poderia ser a resposta ao exemplo de mensagem de requisição que acabamos de discutir.

```
HTTP/1.1 200 OK
Connection: close
Date: Sat, 07 Jul 2007 12:00:15 GMT
Server: Apache/1.3.0 (Unix)
Last-Modified: Sun, 6 May 2007 09:23:24 GMT
Content-Length: 6821
Content-Type: text/html

(data data data data data ...)
```

Vamos examinar cuidadosamente essa mensagem de resposta. Ela tem três seções: uma linha inicial ou **linha de estado**, seis **linhas de cabeçalho** e, em seguida, o **corpo da entidade**, que é o 'filé mignon' da mensagem — contém o objeto solicitado (representado por `data data data data data ...`). A linha de estado tem três campos: o campo de versão do protocolo, um código de estado e uma mensagem de estado correspondente. Nesse exemplo, ela mostra que o servidor está usando o HTTP/1.1 e que está tudo OK (isto é, o servidor encontrou e está enviando o objeto solicitado).

Agora, vamos examinar as linhas de cabeçalho. O servidor usa a linha de cabeçalho `Connection: close` para informar ao cliente que fechará a conexão TCP após enviar a mensagem. A linha de cabeçalho `Date:` indica a hora e a data em que a resposta HTTP foi criada e enviada pelo servidor. Note que esse não é o horário em que o objeto foi criado nem o de sua modificação mais recente; é a hora em que o servidor extraiu o objeto de seu sistema de arquivos, inseriu-o na mensagem de resposta e enviou-a. A linha de cabeçalho `Server:` mostra que a mensagem foi gerada por um servidor Web Apache, análoga à linha de cabeçalho `User-agent:` na mensagem de requisição HTTP. A linha de cabeçalho `Last-Modified:` indica a hora e a data em que o objeto foi criado ou sofreu a última modificação. Esse cabeçalho, que logo estudaremos mais detalhadamente, é fundamental para fazer cache do objeto, tanto no cliente local quanto em servidores de cache da rede (também conhecidos como servidores proxy). A linha de cabeçalho `Content-Length:` indica o número de bytes do objeto que está sendo enviado e a linha `Content-Type:` mostra que o objeto presente no corpo da mensagem é um texto HTML. (O tipo do objeto é oficialmente indicado pelo cabeçalho `Content-Type:` e não pela extensão do arquivo.)

Acabamos de ver um exemplo. Vamos agora examinar o formato geral de uma mensagem de resposta, ilustrado na Figura 2.9.

Esse formato geral de mensagem de resposta condiz com o exemplo anterior. Mas falemos um pouco mais sobre códigos de estado e suas frases, que indicam o resultado da requisição. Eis alguns códigos de estado e frases associadas comuns:

200 OK: requisição bem-sucedida e a informação é entregue com a resposta.

301 Moved Permanently: objeto requisitado foi removido permanentemente; novo URL é especificado no cabeçalho `Location:` da mensagem de resposta. O software do cliente recuperará automaticamente o novo URL.

400 Bad Request: código genérico de erro que indica que a requisição não pôde ser entendida pelo servidor.

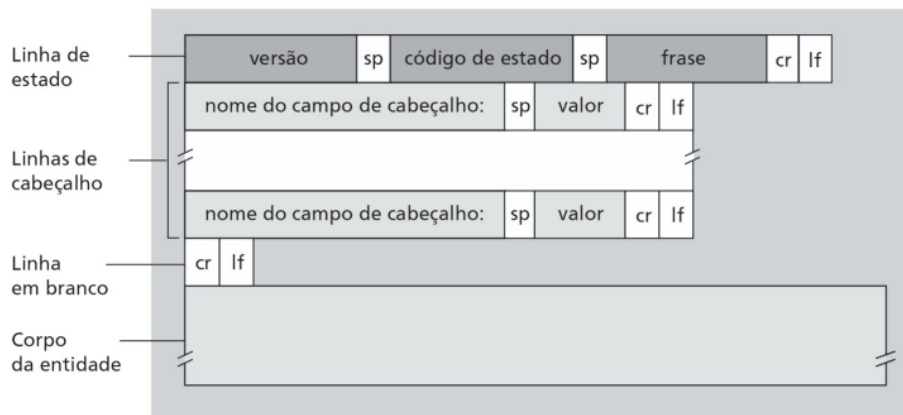


Figura 2.9 Formato geral de uma mensagem de resposta

404 Not Found: o documento requisitado não existe no servidor.

505 HTTP Version Not Supported: a versão do protocolo HTTP requisitada não é suportada pelo servidor.

Você gostaria de ver uma mensagem de resposta HTTP real? É muito recomendável e muito fácil! Primeiramente, dê um comando Telnet em seu servidor Web favorito. Em seguida, digite uma mensagem de requisição de uma linha solicitando algum objeto abrigado no servidor. Por exemplo, se você tem acesso a um prompt de comando, digite:

```
telnet cis.poly.edu 80
```

```
GET /~ross/ HTTP/1.1
```

```
Host: cis.poly.edu
```

(Clique duas vezes a tecla 'enter' após digitar a última linha.) Essa sequência de comandos abre uma conexão TCP para a porta número 80 do hospedeiro `cis.poly.edu` e, em seguida, envia a mensagem de requisição HTTP. Deverá aparecer uma mensagem de resposta que inclui o arquivo-base HTML da homepage do professor Ross. Se você preferir apenas ver as linhas da mensagem HTTP e não receber o objeto em si, substitua GET por HEAD. Finalmente, substitua `/~ross/` por `/~banana/` e veja que tipo de mensagem você obtém.

Nesta seção, discutimos várias linhas de cabeçalho que podem ser usadas em mensagens de requisição e de resposta HTTP. A especificação do HTTP define muitas outras linhas de cabeçalho que podem ser inseridas por browsers, servidores Web e servidores cache de redes. Vimos apenas um pouco do total de linhas de cabeçalho. Examinaremos mais algumas a seguir e mais um pouco quando discutirmos armazenagem Web na Seção 2.2.5. Uma discussão muito abrangente e fácil de ler sobre o protocolo HTTP, seus cabeçalhos e códigos de estado, pode ser encontrada em [Krishnamurty, 2001]; veja também [Luotonen, 1998] se estiver interessado no ponto de vista do desenvolvedor.

Como um browser decide quais linhas de cabeçalho serão incluídas em uma mensagem de requisição? Como um servidor Web decide quais linhas de cabeçalho serão incluídas em uma mensagem de resposta? Um browser vai gerar linhas de cabeçalho em função de seu tipo e versão (por exemplo, um browser HTTP/1.0 não vai gerar nenhuma linha de cabeçalho 1.1), da configuração do usuário para o browser (por exemplo, idioma preferido) e se o browser tem uma versão do objeto possivelmente ultrapassada em sua memória. Servidores Web se comportam de maneira semelhante: há diferentes produtos, versões e configurações, e todos influenciam as linhas de cabeçalho que são incluídas nas mensagens de resposta.

2.2.4 Interação usuário-servidor: cookies

Mencionamos anteriormente que um servidor HTTP não tem estado, o que simplifica o projeto do servidor e vem permitindo que engenheiros desenvolvam servidores Web de alto desempenho que podem manipular milhares de conexões TCP simultâneas. No entanto, é sempre bom que um site Web identifique usuários, seja porque o servidor deseja restringir acesso de usuário, seja porque quer apresentar conteúdo em função da identidade do usuário. Para essas finalidades, o HTTP usa cookies. Cookies, definidos no RFC 2965, permitem que sites monitorem seus usuários. Hoje, a maioria dos sites Web comerciais utiliza cookies.

Como ilustrado na Figura 2.10, a tecnologia dos cookies tem quatro componentes: (1) uma linha de cabeçalho de cookie na mensagem de resposta HTTP; (2) uma linha de cabeçalho de cookie na mensagem de requisição HTTP; (3) um arquivo de cookie mantido no sistema final do usuário e gerenciado pelo browser do usuário; (4) um banco de dados de apoio no site Web. Utilizando a Figura 2.10, vamos esmiuçar um exemplo de como os cookies são usados. Suponha que Susan, que sempre acessa a Web usando o Internet Explorer de seu PC, acesse o Amazon.com pela primeira vez, e que, no passado, ela já tenha visitado o site da eBay. Quando a requisição chega ao servidor Web da Amazon, ele cria um número de identificação exclusivo e uma entrada no seu banco de dados de apoio, que é indexado pelo número de identificação. Então, o servidor Web da Amazon responde ao browser de Susan, incluindo na resposta HTTP um cabeçalho *Set-cookie*: que contém o número de identificação. Por exemplo, a linha de cabeçalho poderia ser:

Set-cookie: 1678

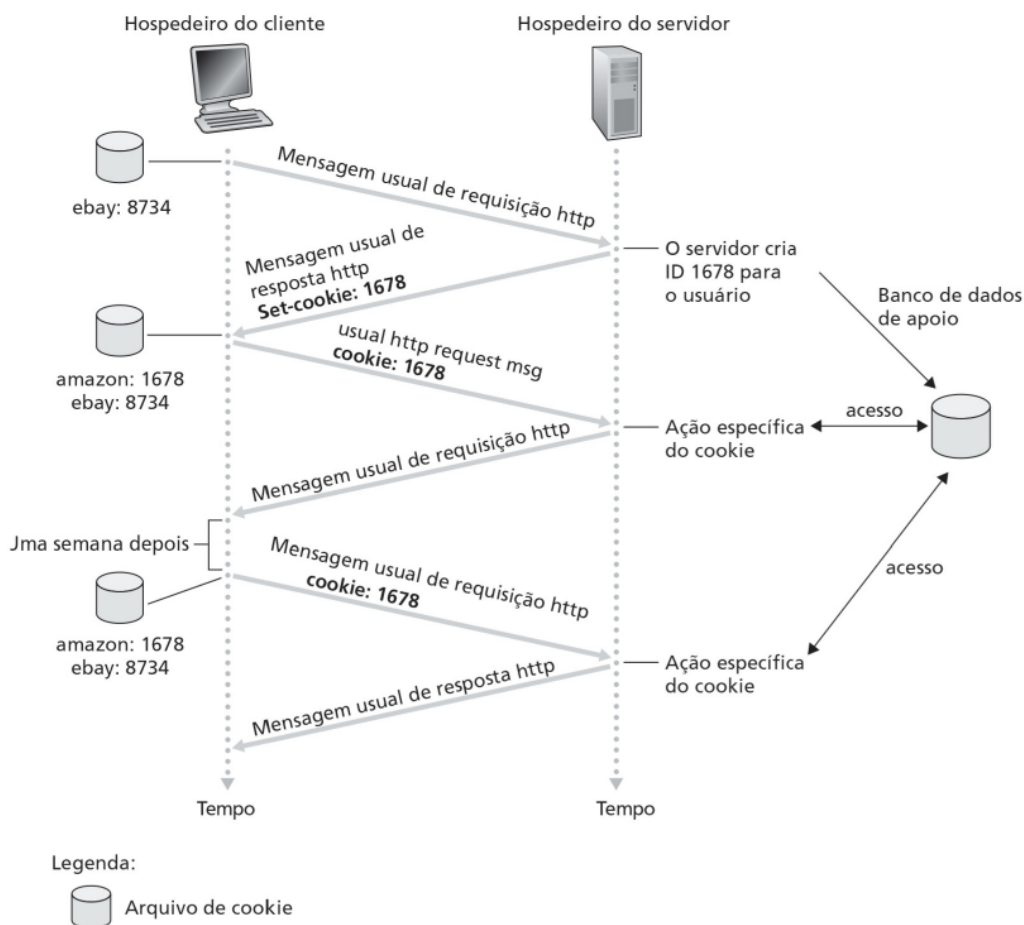


Figura 2.10 Mantendo o estado do usuário com cookies

Quando recebe a mensagem de resposta HTTP, o browser de Susan vê o cabeçalho `Set-cookie:` e, então, anexa uma linha ao arquivo especial de cookies que ele gerencia. Essa linha inclui o nome de hospedeiro do servidor e seu número de identificação nesse cabeçalho. Observe que o arquivo de cookie já possui um entrada para o eBay, uma vez que Susan já visitou esse site no passado. Toda vez que Susan requisita uma página Web enquanto navega pelo site da Amazon em questão, seu browser consulta seu arquivo de cookies, extrai seu número de identificação para esse site e insere na requisição HTTP uma linha de cabeçalho de cookie que inclui o número de identificação. Especificamente, cada uma de suas requisições HTTP ao servidor do site de comércio eletrônico inclui a linha de cabeçalho:

```
Cookie: 1678
```

Dessa maneira, o servidor da Amazon pode monitorar a atividade de Susan e, embora não saiba necessariamente que o nome dela é Susan, sabe exatamente quais páginas o usuário 1678 visitou, em que ordem e em que horários! Então, a Amazon pode utilizar cookies para oferecer um serviço de carrinho de compra — a Amazon pode manter uma lista de todas as compras de Susan, de modo que ela possa pagar por todas elas ao mesmo tempo, no final da sessão.

Se Susan voltar ao site, digamos, uma semana depois, seu browser continuará a inserir a linha de cabeçalho `Cookie: 1678` nas mensagens de requisição. A Amazon pode recomendar produtos a Susan com base nas páginas Web que ela visitou na Amazon anteriormente. Se ela também se registrar no site — fornecendo seu nome completo, endereço de e-mail, endereço postal e informações de cartão de crédito — a Amazon pode incluir essas informações em seu banco de dados e, assim, associar o nome de Susan com seu número de identificação (e com todas as páginas que ela consultou em suas visitas anteriores). É assim que a Amazon e outros sites de comércio eletrônico oferecem “compras com um só clique” — quando quiser comprar um item durante uma visita subsequente, Susan não precisará digitar novamente seu nome, número de seu cartão de crédito, nem endereço.

Essa discussão nos mostrou que cookies podem ser usados para identificar um usuário. Quando visitar um site pela primeira vez, um usuário pode fornecer dados de identificação (possivelmente seu nome). No decorrer das sessões subsequentes, o browser passa um cabeçalho de cookie ao servidor durante todas as visitas subsequentes ao site, identificando, desse modo, o usuário ao servidor. A discussão também deixou claro que cookies podem ser usados para criar uma camada de sessão de usuário sobre HTTP sem estado. Por exemplo, quando um usuário acessa uma aplicação de e-mail baseada na Web (como o Hotmail), o browser envia informações de cookie ao servidor que, por sua vez, identifica o usuário por meio da sessão do usuário com a aplicação.

Embora cookies frequentemente simplifiquem a experiência de compra pela Internet, continuam provocando muita controvérsia porque também podem ser considerados como invasão da privacidade de um usuário. Como acabamos de ver, usando uma combinação de cookies e informações de conta fornecidas pelo usuário, um site Web pode ficar sabendo muita coisa sobre esse usuário e, potencialmente, vender o que sabe para algum terceiro. O Cookie Central [Cookie Central, 2008] inclui informações abrangentes sobre a controvérsia dos cookies.

2.2.5 Caches Web

Um **cache Web** — também denominado **servidor proxy** — é uma entidade da rede que atende requisições HTTP em nome de um servidor Web de origem. O cache Web tem seu próprio disco de armazenagem e mantém, dentro dele, cópias de objetos recentemente requisitados. Como ilustrado na Figura 2.11, o browser de um usuário pode ser configurado de modo que todas as suas requisições HTTP sejam dirigidas primeiramente ao cache Web. Uma vez que esteja configurado um browser, cada uma das requisições de um objeto que o browser faz é primeiramente dirigida ao cache Web. Como exemplo, suponha que um browser esteja requisitando o objeto `http://www.someschool.edu/campus.gif`. Eis o que acontece:

1. O browser estabelece uma conexão TCP com o cache Web e envia a ele uma requisição HTTP para um objeto.
2. O cache Web verifica se tem uma cópia do objeto armazenada localmente. Se tiver, envia o objeto ao browser do cliente, dentro de uma mensagem de resposta HTTP.

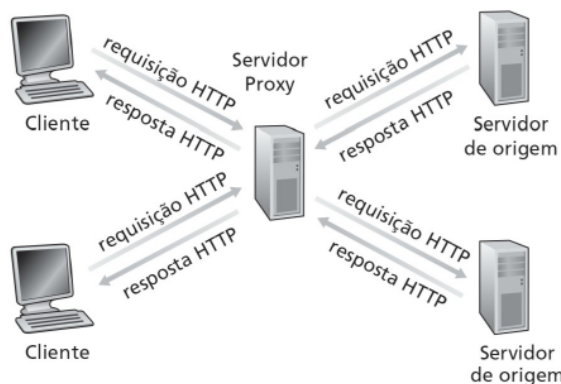


Figura 2.11 Clientes requisitando objetos por meio de um cache Web

3. Se não tiver o objeto, o cache Web abre uma conexão TCP com o servidor de origem, isto é, com `www.someschool.edu`. Então, envia uma requisição HTTP do objeto para a conexão TCP. Após receber essa requisição, o servidor de origem envia o objeto ao cache Web, dentro de uma resposta HTTP.
4. Quando recebe o objeto, o cache Web guarda uma cópia em seu armazenamento local e envia outra, dentro de uma mensagem de resposta, ao browser do cliente (pela conexão TCP existente entre o browser do cliente e o cache Web).

Note que um cache é, ao mesmo tempo, um servidor e um cliente. Quando recebe requisições de um browser e lhe envia respostas, é um servidor. Quando envia requisições para um servidor de origem e recebe respostas dele, é um cliente.

Em geral, é um ISP que compra e instala um cache Web. Por exemplo, uma universidade poderia instalar um cache na rede de seu *campus* e configurar todos os browsers apontando para esse cache. Ou um importante ISP residencial (como a AOL) poderia instalar um ou mais caches em sua rede e configurar antecipadamente os browsers que fornece apontando para os caches instalados.

O cache na Web tem sido utilizado amplamente na Internet por duas razões. Em primeiro lugar, um cache Web pode reduzir substancialmente o tempo de resposta para a requisição de um cliente, em particular se o gargalo da largura de banda entre o cliente e o servidor de origem for muito menor do que aquele entre o cliente e o cache. Se houver uma conexão de alta velocidade entre o cliente e o cache, como em geral é o caso, e se este tiver o objeto requisitado, então ele poderá entregar rapidamente o objeto ao cliente. Em segundo lugar, como logo ilustraremos com um exemplo, caches Web podem reduzir substancialmente o tráfego no enlace de acesso de uma instituição qualquer à Internet. Com a redução do tráfego, a instituição (por exemplo, uma empresa ou uma universidade) não precisa ampliar sua largura de banda tão rapidamente, o que diminui os custos. Além disso, caches Web podem reduzir substancialmente o tráfego na Internet como um todo, melhorando, assim, o desempenho para todas as aplicações.

Para entender melhor os benefícios dos caches, vamos considerar um exemplo no contexto da Figura 2.12. Essa figura mostra duas redes: uma rede institucional e a Internet pública. A rede institucional é uma LAN de alta velocidade. Um roteador da rede institucional e um roteador da Internet estão ligados por um enlace de 15 Mbps. Os servidores de origem estão todos ligados à Internet, mas localizados pelo mundo todo. Suponha que o tamanho médio do objeto seja 1 Mbits e que a taxa média de requisição dos browsers da instituição até os servidores de origem seja de 15 requisições por segundo. Imagine também que o tamanho das mensagens de requisição HTTP seja insignificante e, portanto, elas não criem tráfego nas redes ou no enlace de acesso (do roteador da instituição até o da Internet). Suponha ainda que o tempo entre o envio de uma requisição HTTP (dentro de um datagrama IP) pelo roteador do lado da Internet do enlace de acesso mostrado na Figura 2.12 e o recebimento da resposta (normalmente, dentro de muitos datagramas IPs) seja de 2 segundos em média. Esse último atraso é denominado informalmente 'atraso da Internet'.

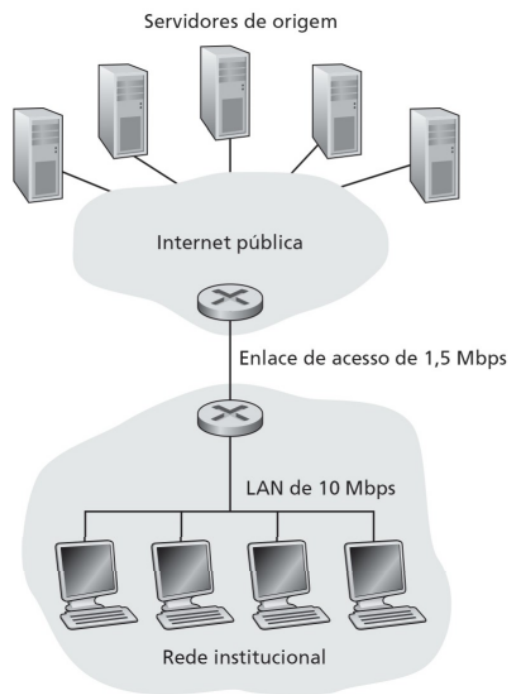


Figura 2.12 Gargalo entre uma rede institucional e a Internet

O tempo de resposta total — isto é, aquele transcorrido entre a requisição de um objeto feita pelo browser e o recebimento dele — é a soma do atraso da LAN, do atraso de acesso (isto é, o atraso entre os dois roteadores) e do atraso da Internet. Vamos fazer agora um cálculo bastante rudimentar para estimar esse atraso. A intensidade de tráfego na LAN (veja a Seção 1.4.2) é

$$(15 \text{ requisições/segundo}) \cdot (1 \text{ Mbits/requisição}) / (100 \text{ Mbps}) = 0,15$$

ao passo que a intensidade de tráfego no enlace de acesso (do roteador da Internet ao da instituição) é

$$(15 \text{ requisições/segundo}) = (1 \text{ Mbits/requisição}) / (15 \text{ Mbps}) = 1$$

Uma intensidade de tráfego de 0,15 em uma LAN resulta em, no máximo, dezenas de milissegundos de atraso; conseqüentemente, podemos desprezar o atraso da LAN. Contudo, como discutimos na Seção 1.4.2, à medida que a intensidade de tráfego se aproxima de 1 (como é o caso do enlace de acesso da Figura 2.12), o atraso em um enlace se torna muito grande e cresce sem limites. Assim, o tempo médio de resposta para atender requisições será da ordem de minutos, se não for maior, o que é inaceitável para os usuários da instituição. Evidentemente, algo precisa ser feito.

Uma possível solução seria aumentar a velocidade de acesso de 15 Mbps para, digamos, 100 Mbps. Isso reduziria a intensidade de tráfego no enlace de acesso a 0,15, o que se traduziria em atrasos desprezíveis entre os dois roteadores. Nesse caso, o tempo total de resposta seria aproximadamente 2 segundos, isto é, o atraso da Internet. Mas essa solução também significa que a instituição tem de atualizar seu enlace de acesso de 15 Mbps para 100 Mbps, o que pode ser muito dispendioso.

Considere agora a solução alternativa de não atualizar o enlace de acesso, mas, em vez disso, instalar um cache Web na rede institucional. Essa solução é ilustrada na Figura 2.13. A taxa de resposta local — a fração de requisições atendidas por um cache — em geral fica na faixa de 0,2 a 0,7 na prática. Para demonstrarmos isso, vamos supor que a taxa de resposta local do cache dessa instituição seja 0,4. Como os clientes e o cache estão conectados à mesma LAN de alta velocidade, 40 por cento das requisições serão atendidas quase

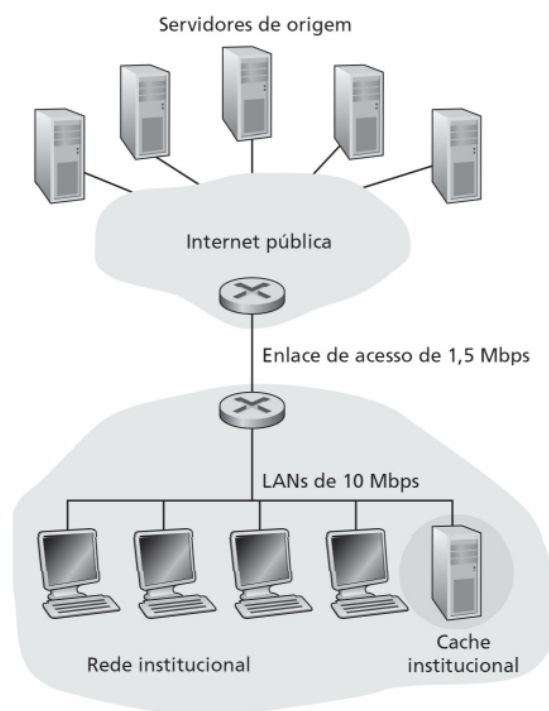


Figura 2.13 Adição de um cache à rede institucional

imediatamente, digamos, em 10 milissegundos. Mesmo assim, os demais 60 por cento das requisições ainda precisam ser atendidos pelos servidores de origem. Mas, com apenas 60 por cento dos objetos requisitados passando pelo enlace de acesso, a intensidade de tráfego neste diminui de 1,0 para 0,6. Em geral, uma intensidade de tráfego menor do que 0,8 corresponde a um atraso pequeno, digamos, dezenas de milissegundos, no caso de um enlace de 15 Mbps. Esse atraso é desprezível se comparado aos 2 segundos do atraso da Internet. Dadas essas considerações, o atraso médio é, por conseguinte, apenas ligeiramente maior do que 1,2 segundo.

$$0,4 \cdot (0,01 \text{ segundo}) + 0,6 \cdot (2,01 \text{ segundos}),$$

Assim, essa segunda solução resulta em tempo de resposta até menor do que o que se conseguiria com a troca do enlace de acesso e não requer que a instituição atualize seu enlace com a Internet. Evidentemente, a instituição terá de comprar e instalar um cache Web. Mas esse custo é baixo — muitos caches usam softwares de domínio público que rodam em PCs baratos.

2.2.6 GET condicional

Embora possa reduzir os tempos de resposta do ponto de vista do usuário, fazer cache introduz um novo problema — a cópia de um objeto existente no cache pode estar desatualizada. Em outras palavras, o objeto abrigado no servidor Web pode ter sido modificado desde a data em que a cópia entrou no cache no cliente. Felizmente, o HTTP tem um mecanismo que permite que um cache verifique se seus objetos estão atualizados. Esse mecanismo é denominado **GET condicional** (*conditional GET*). Uma mensagem de requisição HTTP é denominada uma mensagem GET condicional se (1) usar o método GET e (2) possuir uma linha de cabeçalho *If-Modified-Since*:

Para ilustrar como o GET condicional funciona, vamos examinar um exemplo. Em primeiro lugar, um cache proxy envia uma mensagem de requisição a um servidor Web em nome de um browser requisitante:


```
GET /fruit/kiwi.gif HTTP/1.1
Host: www.exotiquecuisine.com
```

Em segundo lugar, o servidor Web envia ao cache uma mensagem de resposta com o objeto requisitado:

```
HTTP/1.1 200 OK
Date: Sat, 7 Jul 2007 15:39:29
Server: Apache/1.3.0 (Unix)
Last-Modified: Wed, 4 Jul 2007 09:23:24
Content-Type: image/gif
```

```
(data data data data data ...)
```

O cache passa o objeto ao browser requisitante, mas também o guarda em sua memória cache local. O importante é que ele também guarda, juntamente com o objeto, a data da última modificação. Em terceiro lugar, uma semana depois, um outro browser requisita ao cache o mesmo objeto, que ainda está no cache. Como esse objeto pode ter sido modificado no servidor Web na semana anterior, o browser realiza uma verificação de atualização emitindo um GET condicional. Especificamente, o cache envia:

```
GET /fruit/kiwi.gif HTTP/1.1
Host: www.exotiquecuisine.com
If-modified-since: Wed, 4 Jul 2007 09:23:24
```

Note que o valor da linha de cabeçalho `If-modified-since:` é exatamente igual ao valor da linha de cabeçalho `Last-Modified:` que foi enviada pelo servidor há uma semana. Esse GET condicional está dizendo ao servidor para enviar o objeto somente se ele tiver sido modificado desde a data especificada. Suponha que o objeto não tenha sofrido modificação desde 4 Jul 2007 09:23:24. Então, em quarto lugar, o servidor Web envia uma mensagem de resposta ao cache:

```
HTTP/1.1 304 Not Modified
Date: Sat, 14 Jul 2007 15:39:29
Server: Apache/1.3.0 (Unix)
```

```
(corpo de mensagem vazio)
```

Vemos que, em resposta ao GET condicional, o servidor Web ainda envia uma mensagem de resposta, mas não inclui nela o objeto requisitado, o que apenas desperdiçaria largura de banda e aumentaria o tempo de resposta do ponto de vista do usuário, particularmente se o objeto fosse grande. Note que, na linha de estado dessa última mensagem de resposta está inserido 304 Not Modified, que informa ao cache que ele pode seguir adiante e transmitir ao browser requisitante a cópia do objeto que está contida nele.

Finalizamos nossa discussão sobre HTTP, o primeiro protocolo da Internet (um protocolo da camada de aplicação) que estudamos em detalhes. Vimos o formato das mensagens HTTP e as ações tomadas pelo cliente e servidor Web quando essas mensagens são enviadas e recebidas. Também estudamos um pouco da infraestrutura da aplicação da Web, incluindo caches, cookies e banco de dados de apoio, todos associados, de algum modo, ao protocolo HTTP.

2.3 Transferência de arquivo: FTP

Em uma sessão FTP típica, o usuário, sentado à frente de um hospedeiro (o local), quer transferir arquivos de ou para um hospedeiro remoto. Para acessar a conta remota, o usuário deve fornecer uma identificação e uma senha. Após fornecer essas informações de autorização, pode transferir arquivos do sistema local de arquivos para o sistema remoto e vice-versa. Como mostra a Figura 2.14, o usuário interage com o FTP por meio de um agente de usuário FTP. Em primeiro lugar, ele fornece o nome do hospedeiro remoto, o que faz com que o processo cliente FTP do hospedeiro local estabeleça uma conexão TCP com o processo servidor FTP do hospedeiro remoto. O usuário então fornece sua identificação e senha, que são enviadas pela conexão TCP como parte dos